Chapter 1 | Exercise solutions with Python

Tokens, Embeddings, and Neural Language Models

Start with the phrase *the red car*. In Exercise 8, we turn it into four training examples and check which token each context should predict. In Exercise 10, we compare two prefixes and see exactly which part of each prefix the two-token model can use.

To run the checks, open the notebook from a copy of the `transformers-book` repository and select a Python kernel with NumPy. The companion code uses fixed teaching weights, so the cells run on a CPU without a dataset or a training step.

The functions below come from the Chapter 1 example in `examples/ch01_language/model.py`. The first few lines locate that module from the notebook folder or the repository root. You can then run the cells in order.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for folder in (working_directory, *working_directory.parents):
    if (folder / "examples" / "ch01_language" / "model.py").is_file():
        sys.path.insert(0, str(folder))
        break
else:
    raise FileNotFoundError("Open this notebook inside transformers-book.")

from examples.ch01_language.model import (
    TOKEN_TO_ID,
    VOCAB,
    context_logits,
    next_token_pairs,
    stable_softmax,
    toy_parameters,
)
```

8. Build the training pairs

Think of reading *the red car* from left to right. Just before *car*, the two words you can see are *the red*. We pair that input, called the **context**, with *car*, the **label** that gives the correct next token. Each training example needs two input positions. At the very start, we fill missing positions with `BOS` (beginning of sequence). We append `EOS` (end of sequence) so that stopping is also a prediction the model can learn to make.

Write the full sequence as `BOS BOS the red car EOS`. For each label, place a window over the two positions immediately to its left. At the first label, *the*, that window contains two `BOS` tokens. At the next label, *red*, move one position right. The window now contains `BOS` and *the*. Continue until you reach `EOS`.

Context in reading order	Label	Context IDs	Label ID
`BOS BOS`	*the*	`[1, 1]`	4
`BOS` *the*	*red*	`[1, 4]`	12
the red	*car*	`[4, 12]`	7
red car	`EOS`	`[12, 7]`	2

The vocabulary stores the boundary tokens as `<bos>` and `<eos>`. Their IDs are 1 and 2. The window *the red* is paired with the label *car*, and *red car* is paired with `EOS`. These labels come from the reference sentence. This step prepares what the model should learn to predict. It does not call the model or inspect its predictions.

Pass only the sentence's word IDs to `next_token_pairs`. The helper adds the two start tokens and the end token internally. It returns the context array and the label array in matching row order.

```
words = ["the", "red", "car"]
sentence_ids = [TOKEN_TO_ID[word] for word in words]
contexts, targets = next_token_pairs(sentence_ids)

print("Sentence IDs:", sentence_ids)
print(f"{'Context tokens':<17} {'Label':<7} {'Context IDs':<13} Target ID")
for context, target in zip(contexts, targets):
    context_words = " ".join(VOCAB[token_id] for token_id in context)
    label = VOCAB[target]
    print(f"{context_words:<17} {label:<7} "
          f"{str(context.tolist()):<13} {target}")
```

OUTPUT

```
Sentence IDs: [4, 12, 7]
Context tokens    Label   Context IDs   Target ID
<bos> <bos>       the     [1, 1]        4
<bos> the         red     [1, 4]        12
the red           car     [4, 12]       7
red car           <eos>   [12, 7]       2
```

There are four examples, one for each word and one for the ending. Each example has two input IDs and one label ID, so the shapes are `(4, 2)` and `(4,)`. Check the exact order as well as the shapes. A one-position shift could still produce arrays of the expected sizes.

```
expected_contexts = [[1, 1], [1, 4], [4, 12], [12, 7]]
expected_targets = [4, 12, 7, 2]

assert contexts.shape == (4, 2)
assert targets.shape == (4,)
np.testing.assert_array_equal(contexts, expected_contexts)
np.testing.assert_array_equal(targets, expected_targets)
print("All four context windows and labels match the hand calculation.")
```

OUTPUT

```
All four context windows and labels match the hand calculation.
```

Suppose we let the input window for *car* include that occurrence of *car*. A model could learn to copy a token it has already been shown, even though that position will be unknown when it has to generate text. This is **target leakage**. For a next-token example, every context position must come before the target position.

A previous position can hold the same token ID as the target. In the toy sentence *red red car*, the window for the second *red* is `BOS` *red*. The *red* inside this window is the first occurrence. It would already be available when predicting the second. Position tells us whether the information is available, even when two IDs match.

```
repeated_words = ["red", "red", "car"]
repeated_ids = [TOKEN_TO_ID[word] for word in repeated_words]
repeated_contexts, repeated_targets = next_token_pairs(repeated_ids)

np.testing.assert_array_equal(repeated_contexts[1], [1, 12])
assert repeated_targets[1] == 12
print("Context and target for the second red:",
      repeated_contexts[1].tolist(), "->", int(repeated_targets[1]))
```

OUTPUT

```
Context and target for the second red: [1, 12] -> 12
```

10. Find the window's limitation

Imagine continuing *the red* and *I saw the red*. The second prefix has more information for a reader. To see whether this particular network can use it, first convert each complete prefix to IDs. A **prefix** is the sequence available before the token we want to predict. Here each word is one token, and the vocabulary uses a capital `I`.

Prefix	IDs before slicing	IDs given to the model
the red	`[4, 12]`	`[4, 12]`
I saw the red	`[15, 14, 4, 12]`	`[4, 12]`

In Python, `ids[-2:]` starts at the second-to-last item and takes everything to its right, including the starting item. It keeps the two final IDs in their original order. The two earlier IDs 15 and 14 are dropped before the lookup, so they cannot influence the model's calculation.

```
short_prefix = "the red"
long_prefix = "I saw the red"
short_ids = [TOKEN_TO_ID[word] for word in short_prefix.split()]
long_ids = [TOKEN_TO_ID[word] for word in long_prefix.split()]

short_window = short_ids[-2:]
long_window = long_ids[-2:]

assert short_ids == [4, 12]
assert long_ids == [15, 14, 4, 12]
assert short_window == long_window == [4, 12]
print(f"{short_prefix!r}: {short_ids} -> {short_window}")
print(f"{long_prefix!r}: {long_ids} -> {long_window}")
```

OUTPUT

```
'the red': [4, 12] -> [4, 12]
'I saw the red': [15, 14, 4, 12] -> [4, 12]
```

Use the same parameters for both calls. Because the IDs match in both positions, the embedding rows match in the same order. The context layer therefore makes the same **hidden vector**, which is its representation of the window. The output layer receives that vector and makes the same 16 **logits**, one score per vocabulary entry. Softmax then turns each score row into a probability distribution.

From the worked calculation in Exercise 6, we expect the hidden vector $(0, 1, 0.5, 0.3)$, the logit 2 for *car*, the logit 1 for *red*, and zero logits for the other 14 entries. The next cell compares the full arrays from the two calls. It also checks the reference hidden vector, the full row of expected scores, and the sum of the probabilities.

```
parameters = toy_parameters()
short_hidden, short_logits = context_logits(short_window, parameters)
long_hidden, long_logits = context_logits(long_window, parameters)

short_probs = stable_softmax(short_logits)
long_probs = stable_softmax(long_logits)

np.testing.assert_array_equal(short_hidden, long_hidden)
np.testing.assert_array_equal(short_logits, long_logits)
np.testing.assert_array_equal(short_probs, long_probs)

expected_logits = np.zeros(len(VOCAB))
expected_logits[TOKEN_TO_ID["car"]] = 2.0
expected_logits[TOKEN_TO_ID["red"]] = 1.0
np.testing.assert_allclose(short_hidden, [0, 1, 0.5, 0.3])
np.testing.assert_allclose(short_logits, expected_logits)
np.testing.assert_allclose(short_probs.sum(), 1.0)

print("Shared hidden vector:", np.round(short_hidden, 6))
print("Shared logits:", short_logits)
for word in ("car", "red", "the"):
    probability = short_probs[TOKEN_TO_ID[word]]
    print(f"P({word}) = {probability:.6f}")
print("All 16 probabilities match across the two prefixes.")
```

OUTPUT

```
Shared hidden vector: [0.  1.  0.5 0.3]
Shared logits: [0. 0. 0. 0. 0. 0. 0. 2. 0. 0. 0. 0. 1. 0. 0. 0.]
P(car) = 0.306507
P(red) = 0.112757
P(the) = 0.041481
All 16 probabilities match across the two prefixes.
```

The largest probability is about 0.306507 for *car* in both cases. The probability of *red* is about 0.112757. The example prints *the* to illustrate the 0.041481 assigned to **each** of the other 14 entries. The earlier words in the longer prefix made no difference to any score.

To see why this can matter, compare *At the bike shop, I picked the red* … and *At the car dealership, I picked the red* …. The bike shop makes *bike* a natural guess, while the car dealership makes *car* a natural guess. In both phrases, the final two tokens are *the red*. This model would receive the same input even if its weights had been learned from data. The limit comes from which positions the window passes to the network. Within the window, order can still matter, as Exercise 6 showed with *the red* and *red the*.